SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 30**

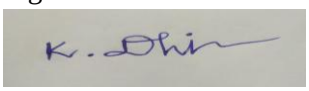
Question Count: 5

Code of Conduct

I **K.Dhilip/192365013** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy,

encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.
5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. **A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.**

1. Probable Distributed Storage System

For billions of records, a distributed storage system such as Hadoop Distributed File System (HDFS) or a cloud object store such as Amazon S3, Azure Data Lake Storage, or Google Cloud Storage would likely be used.

The data would be distributed across multiple servers instead of being stored on one machine.

Why distributed storage?

- Handles extremely large datasets
- Provides horizontal scalability
- Supports parallel processing
- Provides fault tolerance through data replication
- Allows storage capacity to be increased by adding more machines

For example, if the company generates more data next year, additional storage nodes can be added instead of replacing the entire storage system.

2. Data Ingestion Framework

The platform receives data continuously from websites, mobile applications, and transaction systems.

A technology such as Apache Kafka would likely be used for high-volume data ingestion.

For example:

Website Click → Kafka → Data Processing

Mobile Interaction → Kafka → Data Processing

Transaction → Kafka → Data Processing

Kafka can handle large numbers of events and distribute them across multiple topics and partitions**.**

For bulk or scheduled data transfers, tools such as Apache Sqoop or modern cloud-based ingestion services could also be used, depending on the architecture.

3. Separation of Structured and Semi-Structured Data

The platform probably separates data according to its format and usage.

Structured Data

Customer transactions and sales records usually have a fixed structure.

For example:

Customer ID	Product ID	Amount	Date
C101	P501	₹500	20-08-2026

This data could be stored in a data warehouse or structured tables**.**

Semi-Structured Data

Website clicks and mobile events may contain JSON or similar formats.

For example:

```
{
  "user": "C101",
  "page": "products",
  "action": "click",
  "time": "10:30"
}
```

This data could be stored in a data lake in its original form and later transformed for analysis.

This separation makes it easier to process each type of data according to its requirements.

4. Data Partitioning and Indexing

Because billions of records are involved, the data cannot be processed efficiently as one huge dataset.

The system would likely partition data based on fields such as:

- Date
- Region
- Customer ID
- Transaction type
- Website/application

For example:

2026 → August → 20 → Website Clicks

Partitioning allows analytics systems to read only the required portion of the dataset.

Indexing

Structured analytical databases may use indexes on frequently searched fields such as:

- Customer ID
- Transaction ID
- Product ID
- Timestamp

This reduces the amount of data that needs to be scanned during queries.

5. Batch Processing

Historical data needs to be processed periodically for reporting and deeper analytics.

Apache Spark would be a likely batch-processing technology.

For example:

Raw Data → Spark → Cleaning → Transformation → Aggregation → Analytics Database

Spark can divide a large dataset into smaller partitions and process those partitions in parallel across multiple machines.

Older Hadoop-based architectures may also use MapReduce**,** but Spark is generally preferred when faster iterative processing and analytics are required.

6. Stream Processing

Since the platform also requires real-time analytics, a stream-processing engine would likely work alongside the batch-processing system.

Possible technologies include:

- Apache Flink
- Spark Structured Streaming
- Apache Storm

A likely modern architecture would use Kafka + Flink/Spark Structured Streaming**.**

For example:

Customer Click → Kafka → Flink → Real-Time Analytics → Dashboard

This allows the system to detect events almost immediately.

7. Real-Time Analytics

Stream processing could be used for applications such as:

- Live website traffic monitoring
- Real-time sales tracking
- Customer activity monitoring
- Fraud detection
- Recommendation updates
- Real-time alerts

For example, if thousands of users suddenly click on a particular product, the system can detect the increase immediately and display it on an operations dashboard.

8. Scalability Strategy

The architecture most likely uses horizontal scaling.

Instead of purchasing one extremely powerful server, the workload is distributed across many machines.

Example:

1 server → Limited processing

100 servers → Large-scale parallel processing

When the amount of data increases, more nodes can be added to the cluster.

Cloud infrastructure can also provide automatic scaling**,** where additional computing resources are created when workload increases.

9. Partitioning Strategy

Partitioning is particularly important for both storage and processing.

Kafka may partition incoming events according to:

- Customer ID
- Event type
- Region

Distributed storage may partition files according to:

- Date
- Region
- Data type

Processing frameworks can then assign these partitions to different worker nodes.

This enables parallel processing and prevents one machine from becoming a bottleneck.

10. Fault Tolerance

A platform handling billions of transactions cannot depend on individual machines.

The architecture would therefore likely use:

- Data replication
- Multiple processing nodes
- Kafka replication
- Backup storage
- Automatic task recovery
- Cloud availability zones

If one processing node fails, another node can continue processing the workload.

11. Analytics Layer

After processing, cleaned and aggregated data would be placed into an analytics system.

Possible technologies include:

- Cloud data warehouses
- Apache Hive
- BigQuery
- Snowflake
- Amazon Redshift
- Azure Synapse

The analytics layer provides business users with structured information without requiring them to directly query raw big data.

12. Dashboards and Business Intelligence

Business Intelligence tools such as Power BI, Tableau, or Looker would likely connect to the analytics warehouse or a prepared analytics layer.

The architecture could be:

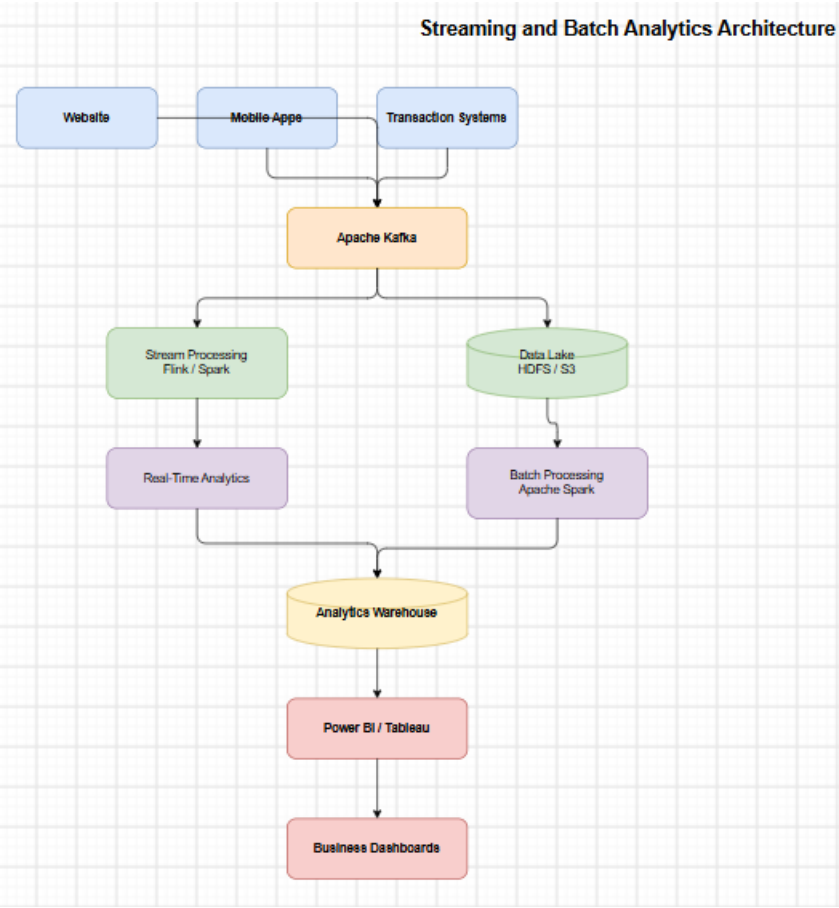
Raw Data → Processing → Data Warehouse → BI Tool → Dashboard

For example, a sales manager could see:

- Total sales
- Website traffic
- Customer activity
- Most popular products
- Regional performance
- Real-time transactions

The BI tools would generally query processed and aggregated datasets rather than repeatedly scanning billions of raw records.

13. Probable Overall Architecture



Conclusion

From the requirements, the platform was most likely designed around distributed storage, high-throughput data ingestion, parallel processing, and real-time stream processing. Technologies such as Kafka, HDFS/cloud object storage, Apache Spark, and Flink or Spark Structured Streaming would be suitable choices.

Structured transaction data would likely be organized into analytical tables, while semi-structured click and mobile-event data could initially be stored in a data lake and transformed when required. Partitioning, replication, horizontal scaling, and parallel processing would allow the platform to handle billions of events reliably.

Finally, processed data would be exposed through a data warehouse or analytics layer**,** allowing BI tools such as Power BI or Tableau to provide dashboards and business reports without directly querying the massive raw datasets.

2. **An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.**

The e-commerce platform handles product views, cart additions, abandoned checkouts, and payments from users across different regions. A scalable big data architecture is likely used to provide real-time personalization.

1.Event Streaming:
Apache Kafka can collect real-time events such as product views, cart updates, and payments. Kafka partitions allow millions of events to be processed simultaneously.

2.Customer Profiles:
A NoSQL database such as Cassandra or MongoDB can store customer profiles and browsing history. Redis can be used as a cache for frequently accessed information and active sessions.

3.Real-Time Processing:
Apache Flink or Spark Streaming can process clickstream and transaction events immediately. This allows customer profiles and recommendations to be updated in real time.

4.Data Storage:
A cloud data lake can store large amounts of raw browsing data, while a **data warehouse** stores cleaned and structured order and payment data.

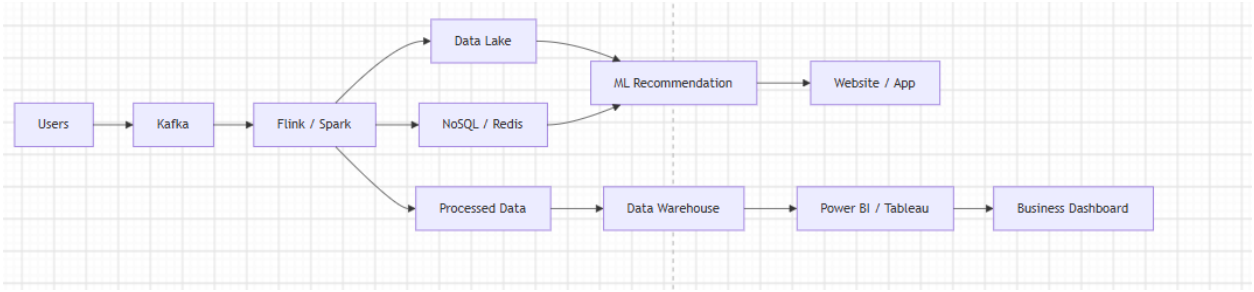
5.Recommendation System:
Machine learning models such as collaborative filtering and content-based filtering analyze customer behavior and recommend relevant products.

6.Data Integration:
Browsing and transaction data can be connected using Customer ID, Product ID, Session ID, and timestamps**.** This helps understand the complete customer journey.

7.Scalability and Fault Tolerance:
The system can use partitioning, database sharding, replication, load balancing, and cloud auto-scaling to handle high traffic and system failures.

8.Dashboards and KPIs:
Processed data can be connected to Power BI or Tableau to monitor sales, conversion rate, cart abandonment, customer activity, and recommendation performance.

Simple Architecture



Conclusion

The system combines Kafka, stream processing, NoSQL databases, Redis, cloud storage, and machine learning to provide fast and personalized shopping experiences. Distributed processing and caching help maintain high performance, scalability, and reliability even with millions of users.

3. **A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.**

The healthcare platform handles patient records, medical images, wearable sensor data, and prescription information**.** A hybrid big data architecture is likely used to manage different types of healthcare data.

1. Hybrid Database Architecture:
Structured data such as patient records and prescriptions can be stored in SQL databases or data warehouses**.** Unstructured data such as X-rays, MRI scans, and medical reports can be stored in a cloud data lake/object storage**.**

2. Data Integration:
Tools such as Apache Kafka, APIs, and ETL tools can connect hospitals, laboratories, pharmacies, and insurance systems. Common healthcare standards such as HL7/FHIR can help different systems exchange data.

3. Streaming and Monitoring:
Wearable devices continuously generate heart rate, blood pressure, and other health data. Kafka with Apache Flink or Spark Streaming can process this data in near real time and generate alerts for abnormal conditions.

4. Machine Learning:
Machine learning models can analyze patient history and sensor data for disease prediction, risk detection, and early diagnosis**.** Technologies such as Apache Spark MLlib or Python-based ML frameworks may be used.

5. Privacy and Security:
Sensitive healthcare information can be protected using encryption, role-based access control, authentication, audit logs, and data masking**.** Access should be limited to authorized healthcare personnel, with applicable healthcare privacy and data-protection regulations enforced.

6. Analytics Pipeline:

Patient Data → Data Integration → Data Lake → Processing → ML/Analytics → Clinical Dashboard

The analytics system can support doctors with patient-risk information and help researchers study disease trends and population health**.**

Conclusion

The platform likely combines SQL databases, cloud data lakes, Kafka, Spark/Flink, and machine learning to manage healthcare data. Strong security, privacy controls, and auditing ensure sensitive patient information is protected while analytics support clinical decisions and population health research.

4. **A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.**

The smart city platform collects traffic, CCTV, weather, and public transport data continuously. A distributed architecture is likely used to process this data in real time.

1. Edge Computing:
Traffic sensors and CCTV systems can use edge devices to process basic information locally. This reduces network traffic and provides faster responses for time-sensitive events.

2. Distributed Messaging:
Apache Kafka can collect and distribute sensor and transport events. Its partitioning allows large amounts of data to be processed in parallel.

3. Centralized Storage:
A cloud data lake can store large volumes of historical sensor data, CCTV metadata, and weather information. A data warehouse can store cleaned data for reporting.

4. Streaming and Geospatial Analytics:
Apache Flink or Spark Streaming can analyze traffic data in real time. Geospatial tools can combine vehicle locations, road networks, and traffic conditions to identify congestion and predict delays.

5. Databases:
A time-series database such as InfluxDB or TimescaleDB can store sensor readings over time.

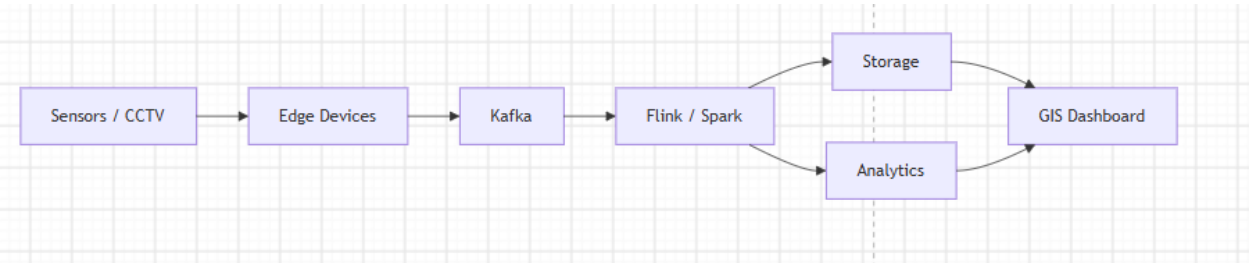
PostGIS/PostgreSQL can support location-based queries such as finding traffic conditions near a particular road or area.

6. Batch + Real-Time Analytics:
Real-time analytics can detect current congestion, while batch processing can analyze historical traffic patterns. Combining both helps city planners improve road design, transport schedules, and traffic management**.**

7. Security:
The system would likely use encryption, authentication, role-based access control, secure APIs, network security, and audit logs**.** Sensitive citizen information should be protected and accessed only by authorized users.

8. Visualization:
Tools such as Power BI, Tableau, or custom GIS dashboards can display live traffic maps, congestion levels, weather conditions, and public transport locations.

Simple Architecture



Conclusion

The platform likely combines edge computing, Kafka, Flink/Spark, cloud storage, time-series databases, and geospatial technologies**.** Real-time processing provides immediate traffic intelligence, while historical analytics supports long-term urban planning. Strong security controls protect citizen and infrastructure data**.**

5. **A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.**

The system combines patient records, wearable readings, lab reports, and appointment data**.** A hybrid architecture is likely used because healthcare data exists in different formats.

1. Hybrid Storage:
Structured data such as patient records and appointments can be stored in SQL databases. Medical reports and other unstructured data can be stored in a cloud data lake/object storage.

2. ETL and Interoperability:
ETL tools can extract, clean, transform, and load data from different hospitals and clinics. HL7/FHIR APIs and standards can help different healthcare systems exchange information consistently.

3. Real-Time Monitoring:
Wearable devices can continuously send health readings through Apache Kafka**.** Flink or Spark Streaming can analyze the data and generate alerts when abnormal readings are detected.

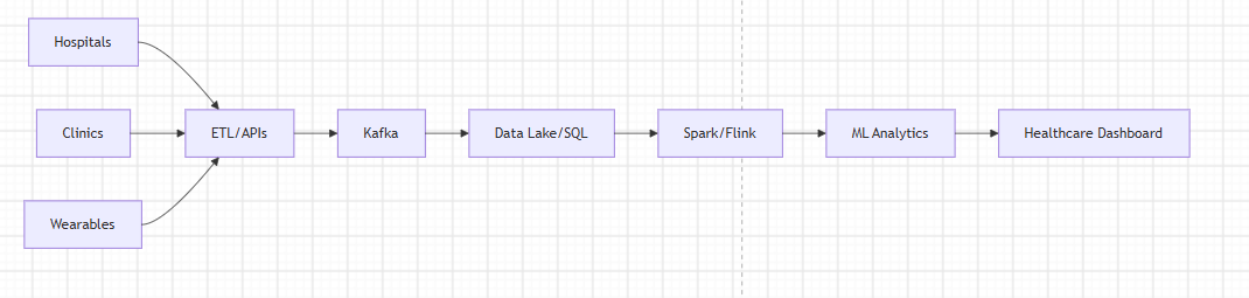
4. Predictive Analytics:
Machine learning models can analyze patient history, lab results, and wearable data to calculate disease or health-risk scores**.** Models can also help identify suitable treatment options, with final decisions remaining under appropriate clinical supervision.

5. Distributed Processing:
Apache Spark can process millions of patient records across multiple machines. This supports population-level studies such as identifying disease trends, risk factors, and healthcare patterns.

6. Security and Compliance:
Sensitive data can be protected using encryption, authentication, role-based access control, data masking, and audit logs**.** The platform should follow applicable healthcare privacy and data-protection regulations.

7. Machine Learning:
ML models can support diagnosis risk scoring, early disease detection, patient monitoring, and treatment recommendations by learning from historical healthcare data.

Simple Architecture



Conclusion

The system likely combines SQL databases, cloud storage, ETL, HL7/FHIR, Kafka, Spark/Flink, and machine learning. This architecture supports real-time patient monitoring and large-scale healthcare analysis while maintaining privacy, security, compliance, and reliable data governance**.**